Logistic Regression: The Perceptron Trick

Contents

1. Overview . . . 2
2. When logistic regression works . . . 2
3. The line and its two sides . . . 3
    3.1 The general form of a line . . . 3
    3.2 Positive and negative sides . . . 4
4. How A, B and C move the line . . . 5
5. The perceptron trick . . . 5
6. Moving the line towards a point . . . 7
    6.1 A negative point on the positive side . . . 7
    6.2 A positive point on the negative side . . . 8
    6.3 Small steps: the learning rate . . . 9
7. Writing the algorithm compactly . . . 11
    7.1 Notation . . . 11
    7.2 Predicting . . . 11
    7.3 One update rule . . . 12
8. Summary . . . 14
9. Sources . . . 14
10. Key terms . . . 14

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Equation of a hyperplane.
- **Leads to:** Logistic regression; Support vector machines; Perceptron; Perceptron loss.

1. Overview

Key point: Logistic regression separates two classes with a straight line. The perceptron trick is a simple way to find such a line: pick a random point, and if it is on the wrong side, move the line towards it.

Logistic regression (G-1120) is one of the most widely used classification algorithms. Logistic regression is also the basic building block of neural networks: a single neuron (the **perceptron**, G-1486) is very close to it. Understanding it well is a good foundation for deep learning.

Logistic regression can be explained in two ways:

- **Geometric:** find a line that separates the classes.
- **Probabilistic:** predict the probability that a point belongs to a class.

These Notes build it step by step. This Note starts with the **perceptron trick** (G-1485), a very simple geometric method. The perceptron trick does not give the best possible line (Bishop §4.1.7), but it explains the idea that the later Notes improve on: the sigmoid function, the loss function and **gradient descent** (G-862).

2. When logistic regression works

Key point: The two classes must be (almost) linearly separable: a straight line, plane or hyperplane can split them.

Take a dataset of students. Each student is one **observation** (G-1374; one record, one row of the data table). There are two **features** (G-772; input variables, one column each), CGPA and IQ, and one **target** (G-1949; the output we predict): placed or not placed. We want a model that takes a new student's CGPA and IQ and predicts whether they will be placed.

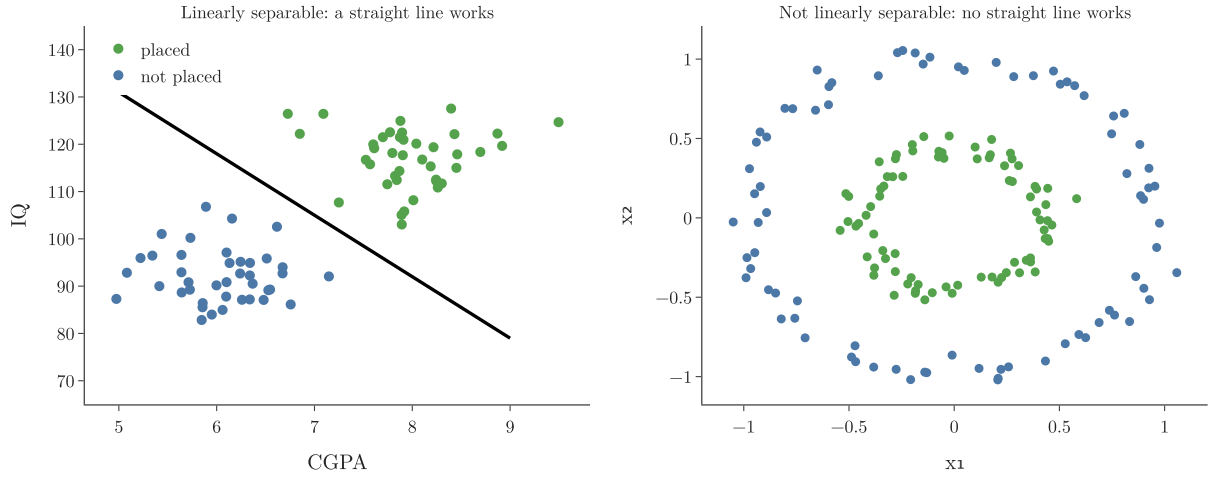


Figure 1: Linearly separable data and data that no straight line can split

In Figure 1 each dot is one observation. In the left panel the green dots (placed) and blue dots (not placed) sit in two groups, and the black line has one group on each side. In the right panel one class forms a ring around the other. Data is **linearly separable** (G-1103) when a straight line can split the two classes (Figure 1, left). With 3 features the divider is a plane, and with more features a **hyperplane** (G-911). The divider between the two classes is the model's **decision boundary** (G-555): points on one side are predicted placed, points on the other side not placed. A few points on the wrong side are fine: the data only needs to be almost separable.

Logistic regression (G-1120) draws a straight divider, just as linear regression fits a straight line. So on data like Figure 1 (right), where one class surrounds the other, it cannot do well.

3. The line and its two sides

> **Key point:** In classification, the line is written $\mathrm{Ax} + \mathrm{By} + \mathrm{C} = 0$. Putting a point into $\mathrm{Ax} + \mathrm{By} + \mathrm{C}$ tells which side it is on.

3.1 The general form of a line

> **Key point:** $\mathrm{Ax} + \mathrm{By} + \mathrm{C} = 0$, or with more features, $\mathrm{w}_0 + \mathrm{w}_1\mathrm{x}_1 + \mathrm{w}_2\mathrm{x}_2 + \ldots = 0$.

Linear regression wrote a line as $y = mx + b$. In classification, both axes are features, so the line is written in its general form:

$$Ax + By + C = 0$$

With CGPA as x_1 and IQ as x_2, this becomes $Ax_1 + Bx_2 + C = 0$. A third feature adds one more term: $Ax_1 + Bx_2 + Cx_3 + D = 0$, a plane. Finding the separating line means finding

good values of A, B and C.

3.2 Positive and negative sides

> **Key point:** Ax + By + C > 0 on one side, < 0 on the other, $= 0$ on the line.

Every line splits the plane into a **positive side** and a **negative side** (G-1529). To find which side a point (p, q) is on, put it into the left-hand side:

- $Ap + Bq + C > 0$: positive side.
- $Ap + Bq + C < 0$: negative side.
- $Ap + Bq + C = 0$: on the line.

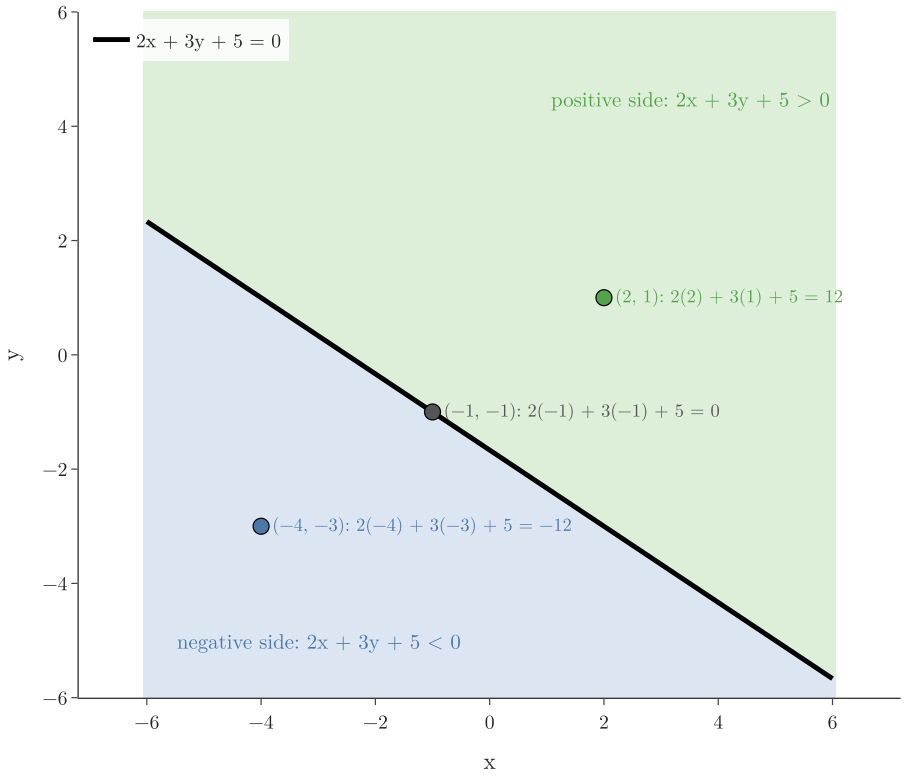


Figure 2: The line 2x + 3y + 5 = 0 and its two sides

With numbers, take the line $2x + 3y + 5 = 0$ (Figure 2). The point $(2, 1)$ gives:

$$2 \times 2 + 3 \times 1 + 5 = 12$$

It is positive, so the point is on the positive side. The point $(-4, -3)$ gives:

$$2 \times (-4) + 3 \times (-3) + 5 = -12$$

It is negative: negative side.

4. How A, B and C move the line

> **Key point:** Changing C shifts the line without turning it. Changing A or B turns it.

The **perceptron trick** (G-1485) moves the line by changing A, B and C. Each one has its own effect (Figure 3):

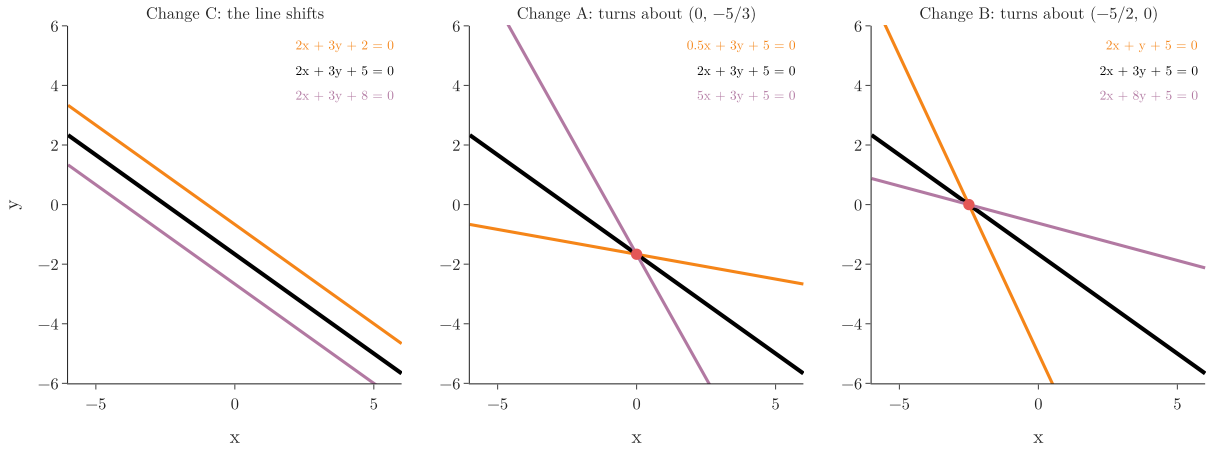


Figure 3: Changing C, A or B, one at a time

- **C:** the line moves parallel to itself. Increasing C moves $2x+3y+C=0$ down, decreasing it moves it up.
- **A:** the line turns about the point where it crosses the y-axis, here $(0, -5/3)$, because there $x = 0$ and A has no effect.
- **B:** the line turns about the point where it crosses the x-axis, here $(-5/2, 0)$.

Changing all three at once combines these moves, so the line can reach any position.

5. The perceptron trick

> **Key point:** Start with any line. Repeatedly pick a random point. If it is on the correct side, do nothing; if not, move the line towards it.

The algorithm:

1. Start with random values of A, B and C: any line.
2. Repeat many times (for example 1,000):
   - pick one training point at random;
   - if it is on the correct side of the line, do nothing;
   - if it is on the wrong side, change A, B and C so that the line moves towards it.
3. Stop after the chosen number of loops, or as soon as no point is misclassified (**convergence**, G-469).

Figure 4 draws the loop.

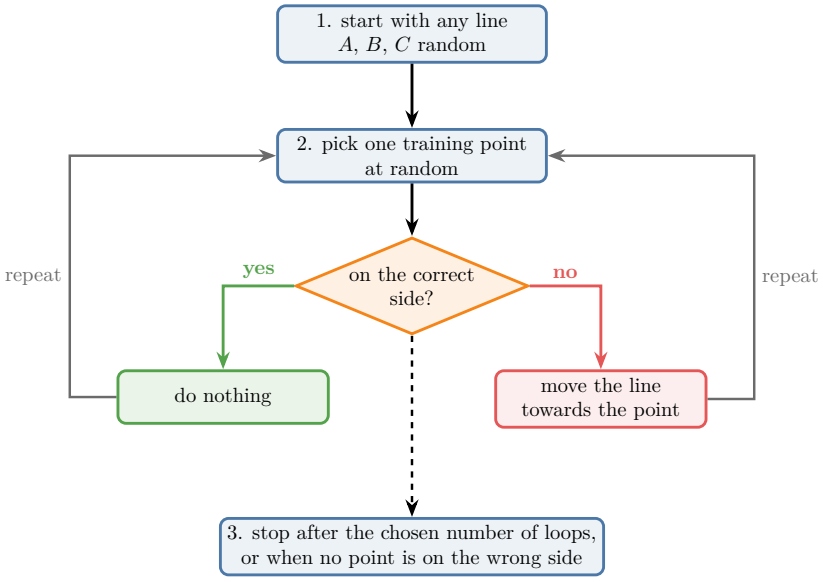


Figure 4: The perceptron trick as a loop: start with any line, pick a random training point, leave the line alone if the point is on the correct side, otherwise move the line towards it; repeat until the chosen number of loops is done or no point is wrong.

Each misclassified point "pulls" the line towards itself until it is on the correct side. After enough pulls, the line settles between the classes.

One way to picture the loop: the line asks each picked student, "are you on the right side?" A correctly classified student answers "yes, leave the line alone". A misclassified student answers "no, come towards me".

Figure 5 runs the loop on 24 students of Figure 1 (12 placed, 12 not placed, both features standardised). Crosses mark misclassified students, and the red ring marks the student picked in that frame. Watch the two kinds of frame. When the ringed student is on the correct side, the line stays. When the ringed student is a cross, the line turns a little towards it (the dashed line is the line before). The start line gets 14 students wrong; after 36 picks, 11 of which moved the line, no student is misclassified and the loop stops.

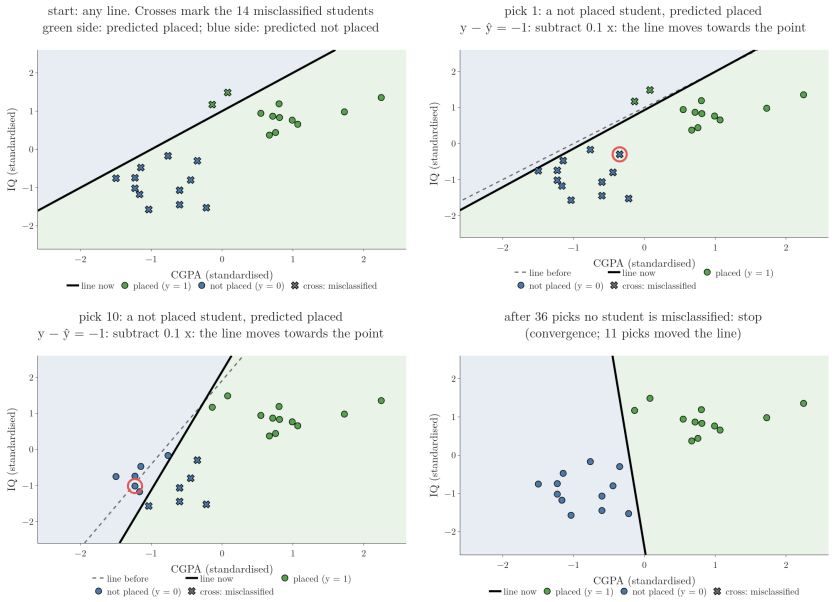


Figure 5: The perceptron trick on 24 students, learning rate 0.1. Each frame picks one student (red ring). Correct side: the line stays. Wrong side (cross): the line moves towards the student; the dashed line is the line before the move. The run stops after 36 picks, when no student is misclassified.

The frame titles also show $y - \hat{y}$, the quantity that Section 7.3 uses to write the whole loop as one rule.

An everyday picture: a farmer builds a fence between sheep and goats. Each time a goat is found on the sheep side, the farmer nudges the fence a little towards that goat; repeated nudges carry the fence past it. Animals already on the right side cause no change. After enough nudges, every animal is on its own side.

6. Moving the line towards a point

> **Key point:** Add a 1 to the point's coordinates. For a negative point on the positive side, subtract them from (A, B, C). For a positive point on the **negative side** (G-1529), add them.

6.1 A negative point on the positive side

> **Key point:** Subtract (x, y, 1) from (A, B, C).

Take the line $2x + 3y + 5 = 0$ and the point $(5, 2)$, which belongs to the negative class. The point gives:

$$2 \times 5 + 3 \times 2 + 5 = 21$$

The value is above 0, so the point is on the positive side: misclassified.

Append a 1 to the point, $(5, 2, 1)$, and subtract it from the coefficients $(2, 3, 5)$:

$$(2-5,\ 3-2,\ 5-1) = (-3,\ 1,\ 4)$$

The new line is:

$$-3x + y + 4 = 0$$

Now the point gives:

$$-3 \times 5 + 1 \times 2 + 4 = -9$$

The value is below 0: it is on the **negative side** (G-1529), as it should be (Figure 6, left).

6.2 A positive point on the negative side

> **Key point:** Add (x, y, 1) to (A, B, C).

The point $(-3, -2)$ belongs to the positive class, but gives a negative value:

$$2 \times (-3) + 3 \times (-2) + 5 = -7$$

Add $(-3, -2, 1)$ instead:

$$(2-3,\ 3-2,\ 5+1) = (-1,\ 1,\ 6)$$

The new line is:

$$-x + y + 6 = 0$$

Now the point gives:

$$-1 \times (-3) + 1 \times (-2) + 6 = 7$$

The value is above 0: positive side (Figure 6, right).

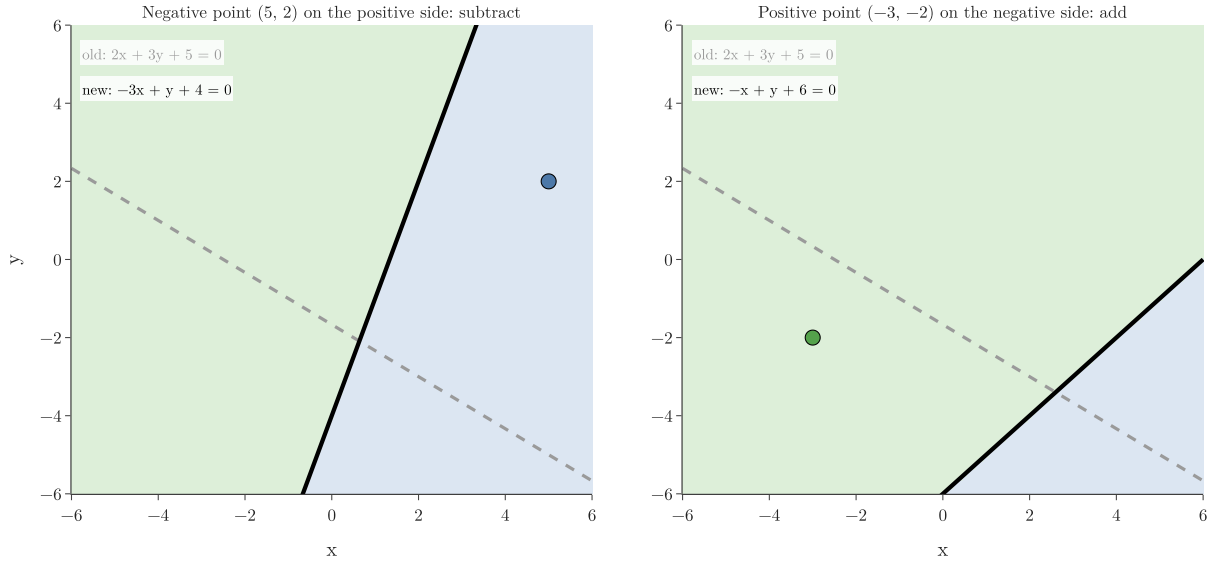


Figure 6: One update for each kind of mistake

6.3 Small steps: the learning rate

> **Key point:** Multiply the point by a learning rate such as 0.1 before adding or subtracting, so the line moves a little at a time.

The full update above jumps the line a long way, which can undo what other points taught it. As in **gradient descent** (G-862), the step is scaled by a small **learning rate** (G-1068) η, usually around 0.1 or 0.01:

$$\text{new coefficients} = \text{old coefficients} - \eta \times (x, y, 1)$$

With $\eta = 0.1$, the first example gives:

$$(2 - 0.5,\ 3 - 0.2,\ 5 - 0.1)$$

$$= (1.5,\ 2.8,\ 4.9)$$

The point's value drops from 21 to 18: the line has moved a little towards it. Repeated small steps get it to the correct side.

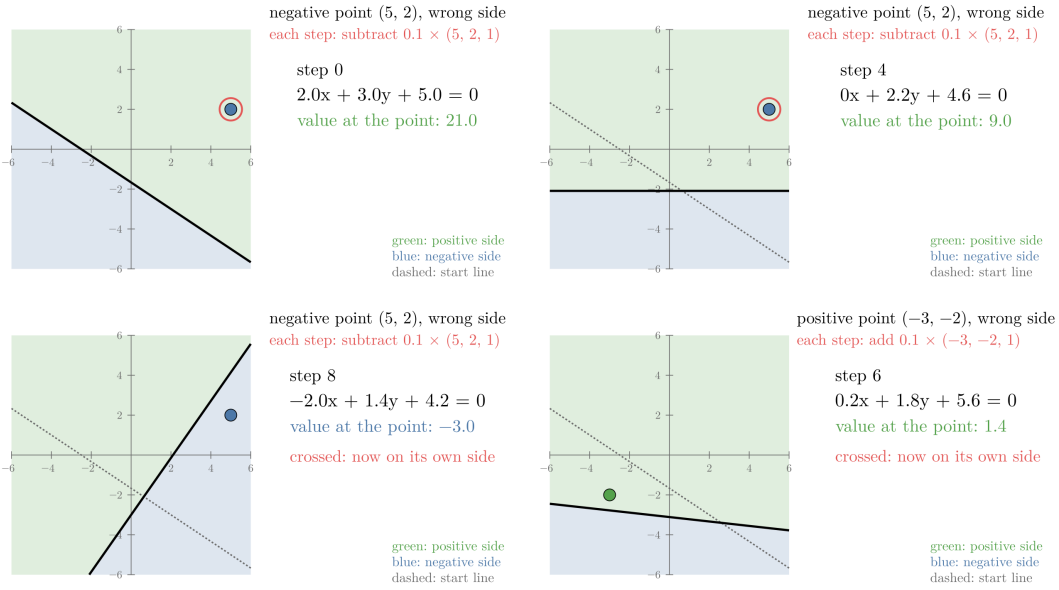


Figure 7: The perceptron trick in small steps, learning rate 0.1. First, the negative point (5, 2) sits on the positive (green) side; each step subtracts $0.1 \times (5, 2, 1)$ from the coefficients, its value falls by 3 (21, 18, 15, ...), and at step 8 the point is on the negative side. Then the positive point $(-3, -2)$; each step adds $0.1 \times (-3, -2, 1)$, its value rises by 1.4 from -7, and it crosses at step 6. The dashed line is the start line $2x + 3y + 5 = 0$

In Figure 7, watch the line swing a little towards the ringed point at every step while the value at the point counts down to 0 and changes sign. Each step changes the value by the same amount. The numbers for the point $(5, 2)$, written as $p = (5, 2, 1)$ with the coefficients $w = (2, 3, 5)$:

$$w \cdot p = 2 \times 5 + 3 \times 2 + 5 \times 1 = 21$$

$$\eta\,(p \cdot p) = 0.1 \times (5 \times 5 + 2 \times 2 + 1 \times 1)$$

$$\eta\,(p \cdot p) = 0.1 \times 30 = 3$$

$$(w - \eta p) \cdot p = 21 - 3 = 18$$

Here $\cdot$ is the **dot product** (G-634): multiply matching entries and add (Section 7.1 spells it out). The new coefficients $(1.5, 2.8, 4.9)$ give the same 18:

$$1.5 \times 5 + 2.8 \times 2 + 4.9 \times 1 = 18$$

For $(-3, -2)$, with $p = (-3, -2, 1)$ and an add step, the change is:

$$0.1 \times (9 + 4 + 1) = 1.4$$

In symbols, the identity behind every step is:

$$(w - \eta p) \cdot p = w \cdot p - \eta(x^2 + y^2 + 1)$$

where $x^2 + y^2 + 1$ is the same $p \cdot p$.

7. Writing the algorithm compactly

> **Key point:** With a column of 1s and weights w, the line is $\Sigma\ \mathrm{w}_i \mathrm{x}_i = 0$, and all four cases fit in one update rule.

7.1 Notation

> **Key point:** Rename C, A, B as w_0, w_1, w_2 and add a column $\mathrm{x}_0 = 1$.

Write the line as $w_0 + w_1 x_1 + w_2 x_2 = 0$, so $w_0 = C$, $w_1 = A$ and $w_2 = B$. Then add an extra feature x_0 that is always 1 (a column of 1s in the data table), as in **multiple linear regression** (G-1279):

$$\sum_{i=0}^{2} w_i x_i = w_0 x_0 + w_1 x_1 + w_2 x_2 = 0$$

The symbol $\sum_{i=0}^{2}$ means "add up the terms for $i = 0, 1, 2$": i is a counter that picks the i-th weight and the i-th feature. A worked instance: the line $2x + 3y + 5 = 0$ has $w = (w_0, w_1, w_2) = (5, 2, 3)$, and the point $(2, 1)$ has $x = (x_0, x_1, x_2) = (1, 2, 1)$:

$$\begin{aligned} i = 0: &\quad w_0 x_0 = 5 \times 1 = 5 \\ i = 1: &\quad w_1 x_1 = 2 \times 2 = 4 \\ i = 2: &\quad w_2 x_2 = 3 \times 1 = 3 \\ &\sum_{i=0}^{2} w_i x_i = 5 + 4 + 3 = 12 \end{aligned}$$

The value 12 is the same as in Section 3.2. The same sum works for any number of features, just with more terms. Multiplying matching entries and adding them is called the **dot product** (G-634), written $w \cdot x$. Here the dot product is 12.

7.2 Predicting

> **Key point:** Compute w $\cdot$ x for the student. If it is positive, predict 1 (placed); otherwise predict 0.

For a student with CGPA 7.5 and IQ 110, the observation is $x = (1,\ 7.5,\ 110)$. Take illustrative weights $w = (-9,\ 0.5,\ 0.05)$. The model computes $w_0 \times 1 + w_1 \times 7.5 + w_2 \times 110$:

$$-9 \times 1 = -9$$

$$0.5 \times 7.5 = 3.75$$

$$0.05 \times 110 = 5.5$$

$$w \cdot x = -9 + 3.75 + 5.5 = 0.25$$

The result 0.25 is above 0, so the model predicts placed (1). In general, if the result is above 0 it predicts placed (1); otherwise not placed (0). The rule "1 if positive, else 0" is a **step function** (G-1889) of $w \cdot x$.

Figure 8 applies it to the three points of Sections 3 and 6 with the line $2x + 3y + 5 = 0$, that is $w = (5, 2, 3)$. The values 12 and 21 land on the step at 1, the value -12 at 0. The point $(5, 2)$ is predicted 1 but belongs to the negative class, so $y - \hat{y} = -1$ in the rule below, and the update subtracts, exactly as in Section 6.1.

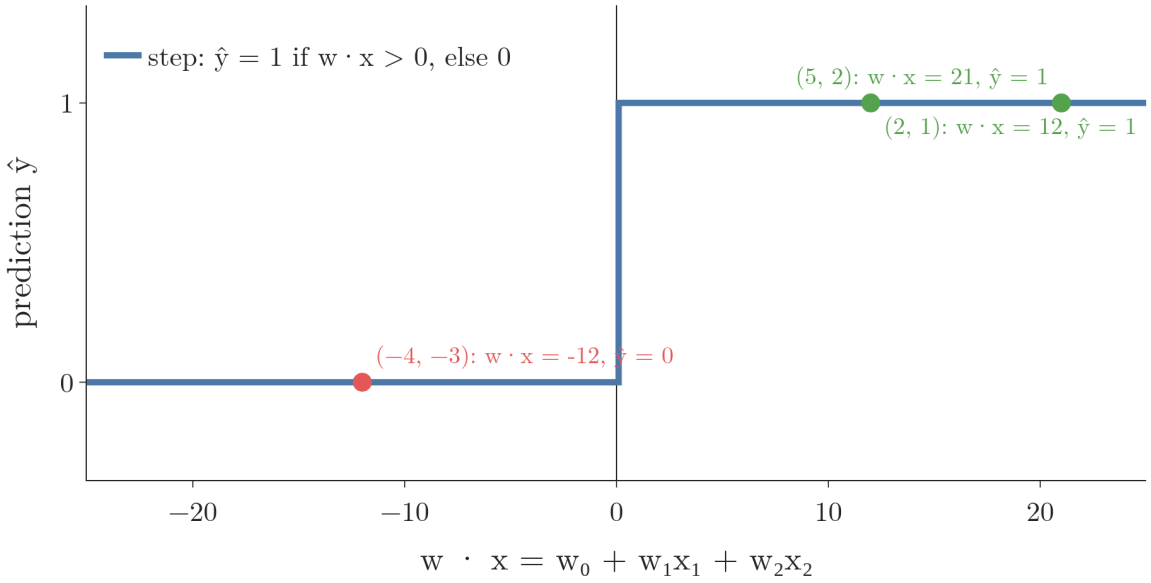


Figure 8: The step function turns w · x into a prediction. With the line 2x + 3y + 5 = 0: (2, 1) gives 12 and $\hat{y} = 1$, (5, 2) gives 21 and $\hat{y} = 1$, (−4, −3) gives −12 and $\hat{y} = 0$.

7.3 One update rule

> **Key point:** w_new = w_old + η (y − $\hat{y}$) x. The factor (y − $\hat{y}$) is 0, +1 or −1 and picks the right action automatically.

The algorithm so far needs two checks: a negative point on the positive side, and a positive point on the **negative side** (G-1529). Both fit into a single rule, where y is the true class and $\hat{y}$ the predicted one:

$$w_{\text{new}} = w_{\text{old}} + \eta\,(y - \hat{y})\,x$$

True y	Predicted $\hat{y}$	$y - \hat{y}$	Update
1	1	0	none: correct
0	0	0	none: correct
1	0	$+1$	$w + \eta x$: add (positive point on the negative side)
0	1	-1	$w - \eta x$: subtract (negative point on the positive side)

So the loop needs no if-statements: for each random point, compute $\hat{y}$ and apply the rule.

Figure 5 shows all four rows at work. Every frame where the line stays is one of the first two rows ($y - \hat{y} = 0$). Picks 1 to 3, where a not-placed student sits on the green side, are the last row (-1, subtract). Picks 4 and 7, where a placed student sits on the blue side, are the third row ($+1$, add).

Python: The **perceptron trick** (G-1485).

```python
def step(z):
    return 1 if z > 0 else 0

def perceptron(X, y, lr=0.1, loops=1000):
    X = np.insert(X, 0, 1, axis=1)      # column of 1s
    w = np.ones(X.shape[1])
    for _ in range(loops):
        j = np.random.randint(0, X.shape[0])
        y_hat = step(X[j] @ w)
        w = w + lr * (y[j] - y_hat) * X[j]
    return w
```

Running this code on real data, we can watch the decision boundary move.

8. Summary

- Logistic regression needs (almost) **linearly separable** (G-1103) classes, because it draws a straight divider; data where one class surrounds the other defeats it.
- A line $Ax + By + C = 0$ has a positive and a **negative side** (G-1529); plug a point in to find which, so the sign of the value is the model's prediction.
- **Perceptron trick** (G-1485): pick random points; move the line towards each misclassified one, so each mistake pulls the line until that point is on its correct side.
- Update: $w \leftarrow w + \eta(y - \hat{y})x$, with a column of 1s in x; because $y - \hat{y}$ is 0, +1 or −1, one rule covers do nothing, add and subtract, and the small η keeps one point from undoing what others taught.
- The perceptron trick finds a separating line, but not necessarily the best one; which line it finds depends on the starting line and the order of the points (Bishop §4.1.7). The next Notes fix this.

9. Sources

Built from

- CampusX, "Logistic Regression Part 1 | Perceptron Trick", YouTube, https://www.youtube.com/watch

Other references

- **Bishop:** Bishop, C. M. *Pattern Recognition and Machine Learning*. Springer, 2006. Section 4.1.7, pp. 192–196 (the perceptron algorithm; p. 194 on many solutions).

10. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Logistic regression (G-1120)	A classification algorithm that passes a weighted sum of the inputs through the sigmoid to give the probability of a class; the line (or plane) where that probability is 0.5 is the boundary it learns between the classes.
Perceptron trick (G-1485)	Moving a line towards each misclassified point until the classes are separated.
Linearly separable (G-1103)	Data whose classes a straight line, plane or hyperplane can split.
Positive and negative side (G-1529)	The two halves of the plane on either side of a line $Ax + By + C = 0$: where $Ax + By + C$ is above 0 and where it is below 0; the perceptron predicts one class on each side.

Term	Meaning
Step function (G-1889)	A function that outputs 1 for positive inputs and 0 otherwise; the classic perceptron uses it to turn its score into a class.
Perceptron (G-1486)	The smallest building block of a neural network, one artificial neuron: it multiplies each input by a weight, adds the results and turns the sum into an output.
Gradient descent (G-862)	Finding the lowest point of a function by repeated small steps downhill.
Observation (G-1374)	One record of a dataset: one row of the data table.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
Target (G-1949)	The output column we predict, such as the class; also called the label.
Hyperplane (G-911)	A flat surface in more than three dimensions; the model for three or more input columns.
Decision boundary (G-555)	A line or curve that separates the classes in classification.
Converge (G-469)	To settle at a minimum, with steps becoming negligible.
Learning rate (η) (G-1068)	How strongly each update changes the model; in gradient descent, the number the slope is multiplied by to get the step size.
Dot product (G-634)	Multiplying two vectors entry by entry and adding the products, $u^{\mathsf{T}}x$, which gives one number. It is large when the vectors point the same way, so it measures similarity and gives projections.
Multiple linear regression (G-1279)	Linear regression with several input columns.